

L6 Practice Guide: Shared Memory and Synchronization

Prepared from your `sop-keys.c` and git stages

April 15, 2026

How to use this file before lab

- Read Stage 1–4 in order; each stage extends previous logic.
- Keep `lab6_helpers.c` open as your “copy-ready” toolbox.
- In exam conditions, start with the skeleton checklist in Section “Fast build order”.
- If stuck, prioritize correct synchronization over print format polish.

Task model recap

The lab models students cleaning keyboards:

- There are m keyboards, each keyboard has k keys.
- At most `KEYBOARD_CAP` students can stand at a keyboard at once.
- Work is done by n child processes.
- Synchronization mechanisms used over stages:
 - named semaphores (`/sop-sem-i`),
 - process-shared barrier in anonymous shared memory,
 - named shared memory with keyboard dirt values (`double`),
 - process-shared robust mutex per key,
 - shared panic flag to stop everyone after accident detection.

Fast build order (exam-safe checklist)

1. Parse and validate `n m k`.
2. Unlink old semaphores before starting (`ENOENT` is okay).
3. Create anonymous shared block for synchronization primitives.
4. Initialize barrier with count `n+1`.
5. Initialize process-shared mutex attributes; add robust mode for Stage 4.
6. Fork children.
7. Parent creates and initializes named shared-memory array to `1.0`.
8. Parent sleeps 500ms and then enters barrier.
9. Children pass barrier, open semaphores, map keyboard array, and clean.
10. Parent waits for all children, prints final state, unlinks shared objects.

Stage 1: 1 (Semaphores + fork)

Goal: limit simultaneous access to each keyboard by capacity with named semaphores.

What your stage-1 commit got right

- Removing stale semaphores at start.
- Each child opening all keyboard semaphores.
- Random keyboard selection and `sem_wait` / `sem_post` around cleaning.
- Parent waiting for all children and cleaning semaphores again at end.

Key excerpt from your solution path

```
1 void child_work(int m)
2 {
3     char sem_name[20];
4     sem_t *sem_arr[MAX_KEYBOARDS];
5     srand(time(NULL)^getpid());
6     for (int i = 0; i < m; i++) {
7         snprintf(sem_name, sizeof(sem_name), "/sop-sem-%d", i);
8         sem_arr[i] = sem_open(sem_name, O_CREAT, 0666, KEYBOARD_CAP);
9     }
10    for (int i = 0; i < 10; i++) {
11        int rand_sem = rand() % m;
12        sem_wait(sem_arr[rand_sem]);
13        printf("Student %d: cleaning keyboard %d\n", getpid(), rand_sem);
14        ms_sleep(300);
15        sem_post(sem_arr[rand_sem]);
16    }
17 }
```

Stage-1 reasoning template

1. Semaphore value encodes “free standing spots” near keyboard.
2. `sem_wait` blocks if keyboard is full.
3. Critical region is the cleaning time window (300ms).
4. `sem_post` returns one slot.

Common mistakes to avoid

- Forgetting to unlink old semaphore names before run.
- Unbalanced `sem_wait`/`sem_post` on error paths.
- Using one semaphore for all keyboards instead of one per keyboard.

Stage 2: 2 (Barrier in anonymous shared memory)

Goal: children must not start semaphore logic before parent is ready.

What changed from Stage 1

- Added shared anonymous memory block (`MAP_SHARED|MAP_ANON`).
- Put `pthread_barrier_t` inside that block.
- Barrier count is `n+1` (all children + parent).
- Child waits at barrier before opening semaphores.
- Parent sleeps 500ms after forking, then enters same barrier.

Key excerpt from your solution path

```
1 typedef struct shared {
2     pthread_barrier_t barrier;
3 } shared_t;
4
5 shared_t* shared = mmap(NULL, sizeof(shared_t),
6                          PROT_READ|PROT_WRITE,
7                          MAP_SHARED|MAP_ANON, -1, 0);
8
9 pthread_barrierattr_t barrier_attr;
10 pthread_barrierattr_init(&barrier_attr);
11 pthread_barrierattr_setpshared(&barrier_attr, PTHREAD_PROCESS_SHARED);
12 pthread_barrier_init(&shared->barrier, &barrier_attr, n + 1);
13
14 /* child side */
15 pthread_barrier_wait(&shared->barrier);
16
17 /* parent side */
18 ms_sleep(500);
19 pthread_barrier_wait(&shared->barrier);
```

Why this solves Stage 2 exactly

Children cannot pass `pthread_barrier_wait()` until parent also arrives. This guarantees a deterministic “all-start-together” point and prevents child access to yet-unprepared shared resources in later stages.

Stage 3: 3 (Named shared memory + per-key mutex)

Goal: model keyboard dirt values and synchronize key-level updates.

Required architecture

- Named memory object stores `m*k` doubles.
- Parent creates object with `shm_open` + `ftruncate` + `mmap`.
- Parent initializes all values to 1.0.
- Children map same object after barrier.
- Every clean event: choose keyboard and key, sleep 300ms, divide selected field by 3.
- Add key index to output message.
- Protect each key with a dedicated mutex.

Excerpt (mapping and initialization pattern)

```
1 int fd = shm_open(SHARED_MEM_NAME, O_CREAT | O_RDWR, 0666);
2 ftruncate(fd, m * k * sizeof(double));
3 double* shared_mem = mmap(NULL, m * k * sizeof(double),
4                             PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
5
6 for (int i = 0; i < m * k; i++) {
7     shared_mem[i] = 1.0;
8 }
```

Recommended indexing pattern

Use a flat index:

$$\text{idx} = \text{keyboard_id} * k + \text{key_id}$$

Then lock `key_mutex[idx]`, update `shared_mem[idx]`.

Important correction compared to your current final file

For Stage 3/4 correctness, mutex and update should target the selected *key*, not only keyboard id. Also ensure each mutex lock has exactly one matching unlock.

Stage 4: 4 (Infinite loop + accidents + robust mutex)

Goal: handle process crash while holding mutex and trigger global panic.

Mechanics required by task

- Cleaning loop is now infinite.
- Before key update, with probability 1%, student prints exhaustion message, releases semaphore, and `abort()`.
- Other student locking same robust mutex receives `EOWNERDEAD`.
- That student prints “someone is lying here, help!!!”, sets shared panic flag, and all students stop loop.

Excerpt from your Stage-4 style logic

```
1 if (pthread_mutex_lock(&shared->keyboards[rand_sem]) == EOWNERDEAD) {
2     pthread_mutex_lock(&shared->find_dead_mutex);
3     shared->find_dead = 1;
4     printf("Student <%d>: someone is lying here, help!!!\n", getpid());
5     pthread_mutex_unlock(&shared->find_dead_mutex);
6     pthread_mutex_consistent(&shared->keyboards[rand_sem]);
7 }
```

Robust mutex interpretation

- `EOWNERDEAD` means previous owner died while holding lock.
- You must call `pthread_mutex_consistent()` before unlock.
- After setting shared panic flag, all processes should check flag regularly and exit loop quickly.

Practical exam notes

- Crash chance: use `rand() % 100 == 0` for exact 1%.
- Release semaphore before `abort()` as task states.
- Keep panic flag under a dedicated mutex to avoid races.

Resource cleanup order

Use this order to reduce leaks and weird rerun behavior:

1. Children: close semaphores, unmap keyboard memory, close shm fd.
2. Parent: wait all children.
3. Parent: print final keyboard state.
4. Parent: unlink semaphores and named shared memory.
5. Parent: destroy barrier and mutexes in anonymous shared block.
6. Parent: unmap anonymous shared block.

Minimal command cheatsheet

```
1 gcc -std=c11 -Wall -Wextra -O2 sop-keys.c -o sop-keys -pthread -lrt
2 ./sop-keys 10 3 8
```

Some systems do not need `-lrt` for `shm_open`; keeping it is usually safe on Linux labs.

Final tactical advice for tomorrow

- Build incrementally: finish Stage 1 completely before adding barrier.
- Test with small values first (e.g., `n=10`, `m=2`, `k=5`).
- Keep one “flat key index” helper to avoid indexing bugs.
- If output is noisy, correctness of synchronization still matters more.
- If time pressure hits, prioritize: barrier + shm init ordering + robust mutex path.